SGX 하드웨어 명령어 소프트웨어 래퍼 구현 분석

개요

이 문서는 Intel SGX SDK에서 SGX 하드웨어 명령어들의 소프트웨어 래퍼 구현을 상세히 분석합니다. SGX 하드웨어는 ENCLU(Enclave User) 및 ENCLS(Enclave Supervisor) 명령어 세트를 제공하며, 이들 명령어는 소프트웨어에서 어셈블리 인라인 코드와 시스템 호출을 통해 래핑됩니다.

1. ENCLU 명령어 래퍼 구현 분석

1.1 기본 ENCLU 매크로 구현

파일: `common/inc/internal/linux/linux-regs.h`

```
.macro ENCLU

#ifdef SE_SIM

cmp $SE_EEXIT, %xax

jne 1f


/* if leaf is EEXIT, xbp and xsp need to be passed by xdx and xcx */

mov %xbp, %xdx

mov %xsp, %xcx

1:

push %xdi

push %xsi
```

```
push %xdx

push %xcx

push %xbx

push %xax


# ifdef LINUX64

pop %rdi

pop %rsi

pop %rdx

pop %rcx

pop %r8

pop %r9

# endif


.type _SE3,@function

.protected _SE3

call _SE3


# ifdef LINUX32

add $(SE_WORDSIZE * 6), %esp

# endif


#else /* SE_SIM */
```

```
.byte 0x0f, 0x01, 0xd7 /* 실제 ENCLU 명령어 바이트코드 */
#endif /* !SE_SIM */
.endm
```

핵심 분석 포인트:

- **하드웨어 모드**: `0x0f, 0x01, 0xd7` 바이트코드로 직접 ENCLU 명령어 실행
- **시뮬레이션 모드**: `_SE3` 함수 호출로 소프트웨어 에뮬레이션
- **레지스터 보존**: 스택을 통한 레지스터 상태 보존 및 복원
- **플랫폼별 처리**: LINUX32/LINUX64에 따른 다른 레지스터 처리

1.2 EENTER 명령어 래퍼

파일: `psw/urts/linux/enter_enclave.S`

```
.Ldo_eenter:
# clean the upper bits of YMM registers
lea_symbol g_clean_ymm, %xbx
movl (%xbx), %ecx
cmpl $0, %ecx
je 1f
vzeroupper
1:
mov frame_arg0, %xbx /* tcs addr */
lea_pic .Lasync_exit_pointer, %xcx /* aep addr */
mov $SE_EENTER, %xax /* EENTER leaf */
```

```
.Leenter_inst:
ENCLU
```

핵심 분석 포인트:

- **레지스터 설정**: RAX=SE_EENTER(2), RBX=TCS 주소, RCX=AEP 주소
- **YMM 레지스터 정리**: `vzeroupper` 로 상위 비트 클리어
- **AEP(Asynchronous Exit Pointer)**: 비동기 종료 시 복귀 주소 설정

1.3 EEXIT 명령어 래퍼

파일: `sdk/trts/linux/trts_pic.S`

```
.Lclear_and_exit_enclave:
/* Clear all GPRs, except xax, xbx, xdi and xsi */
xor %xcx, %xcx
xor %xdx, %xdx
#if defined(LINUX64)
xor %r8, %r8
xor %r9, %r9
xor %r10, %r10
xor %r11, %r11
xor %r12, %r12
xor %r13, %r13
xor %r14, %r14
```

```
xor %r15, %r15
#endif


/* Set status flags to pre-defined values */
add %xdx, %xdx /* OF = SF = AF = CF = 0; ZF = PF = 1 */
cld /* DF = 0 */


/* EEXIT */
mov $SE_EEXIT, %xax /* EEXIT leaf */
ENCLU
```

핵심 분석 포인트:

- **레지스터 정리**: 보안을 위해 모든 범용 레지스터 클리어
- **플래그 초기화**: 상태 플래그를 미리 정의된 값으로 설정
- **정보 누출 방지**: Enclave 내부 정보가 외부로 유출되지 않도록 보장

1.4 EGETKEY 명령어 래퍼

파일: `sdk/trts/linux/trts_pic.S`

```
DECLARE_LOCAL_FUNC do_egetkey
SE_PROLOG
mov $SE_EGETKEY, %xax /* EGETKEY leaf */
ENCLU
#ifdef SE_SIM
```

```
cmp $SGX_SUCCESS, %xax /* In simulation mode, ZF flag will not be set */
jnz .Legetkey_done /* because the stack clean operation will always clean ZF fla
#else
jz .Legetkey_done /* if EGETKEY error, ZF flag is set and error code is set to x
#endif
xor %xax, %xax
.Legetkey_done:
SE_EPILOG
END_FUNC
```

핵심 분석 포인트:

- **레지스터 규약**: RBX=key_request 주소, RCX=key 출력 주소
- **에러 처리**: ZF 플래그로 성공/실패 판단, RAX에 에러 코드
- **시뮬레이션 모드 차이**: ZF 플래그 신뢰성 문제로 반환 코드 직접 확인

1.5 EREPORT 명령어 래퍼

파일: `sdk/trts/linux/trts_pic.S`

```
.global Lereport_inst
DECLARE_LOCAL_FUNC do_ereport
SE_PROLOG
mov $SE_EREPORT, %xax /* EREPORT leaf */
clc
Lereport_inst:
```

```
ENCLU

setc %al

SE_EPILOG

END_FUNC
```

핵심 분석 포인트:

- **레지스터 규약**: RBX=target_info, RCX=report_data, RDX=report 출력
- **에러 처리**: Carry flag로 성공/실패 판단
- **예외 처리**: `Lereport_inst` 레이블로 예외 핸들러에서 참조

2. ENCLS 명령어 래퍼 구현 분석

2.1 커널 드라이버 인터페이스

ENCLS 명령어는 특권 모드에서만 실행 가능하므로 커널 드라이버를 통해 처리됩니다.

파일: `psw/urts/linux/isgx_user.h`

```
/**
* struct sgx_enclave_create - parameter structure for the
* %SGX_IOC_ENCLAVE_CREATE ioctl
* @src: address for the SECS page data
*/
struct sgx_enclave_create {
__u64 src;
};
```

```
/**
* struct sgx_enclave_add_pages - parameter structure for the
* %SGX_IOC_ENCLAVE_ADD_PAGES ioctl
* @src: start address for the page data
* @offset: starting page offset
* @length: length of the data (multiple of the page size)
* @secinfo: address for the SECINFO data
* @flags: page control flags
* @count: number of bytes added (multiple of the page size)
*/
struct sgx_enclave_add_pages {
__u64 src;
__u64 offset;
__u64 length;
__u64 secinfo;
__u64 flags;
__u64 count;
};
```

2.2 IOCTL 시스템 호출 매핑

```
#define SGX_IOC_ENCLAVE_CREATE \
```

```
_IOW(SGX_MAGIC, 0x00, struct sgx_enclave_create)

#define SGX_IOC_ENCLAVE_ADD_PAGES \

_IOWR(SGX_MAGIC, 0x01, struct sgx_enclave_add_pages)

#define SGX_IOC_ENCLAVE_INIT \

_IOW(SGX_MAGIC, 0x02, struct sgx_enclave_init)
```

핵심 분석 포인트:

- **ECREATE**: `SGX_IOC_ENCLAVE_CREATE` IOCTL로 SECS 생성
- **EADD**: `SGX_IOC_ENCLAVE_ADD_PAGES` IOCTL로 페이지 추가
- **EINIT**: `SGX_IOC_ENCLAVE_INIT` IOCTL로 Enclave 초기화
- **커널 검증**: 드라이버에서 매개변수 검증 및 권한 확인

2.3 vDSO 인터페이스

파일: `psw/urts/linux/enter_enclave.S`

```
DECLARE_GLOBAL_FUNC vdso_sgx_enter_enclave_wrapper

EENTER_PROLOG

movl frame_arg0, %edi

#if defined(__x86_64__)

/* we defined fn as int, so we do sign extend.*/

movslq %edi, %rdi

#endif

mov frame_arg1, %xsi

mov frame_arg2, %xdx
```

```
# clean the upper bits of YMM registers

lea_symbol g_clean_ymm, %xbx

movl (%xbx), %ecx

cmpl $0, %ecx

je 1f

vzeroupper

1:

sub $(1*SE_WORDSIZE), %xsp

mov 2*SE_WORDSIZE(%xbp), %xcx

push %xcx

mov frame_arg3, %xcx

lea_pic vdso_sgx_enter_enclave, %r10

call *(%r10)

mov %xax, %xsi

EENTER_EPILOG

END_FUNC
```

핵심 분석 포인트:

- **vDSO 활용**: 커널 모드 전환 없이 SGX 명령어 실행
- **성능 최적화**: 시스템 호출 오버헤드 감소
- **예외 처리**: vDSO에서 예외 상황 처리 및 복구

3. 에러 처리 메커니즘 분석

3.1 하드웨어 예외를 소프트웨어 에러 코드로 변환

파일: `sdk/trts/trts_veh.cpp`

```
if (size_t(&Lereport_inst) == ssa_gpr->REG(ip) && SE_EREPORT == ssa_gpr->REG(ax)
{
// Handle the exception raised by EREPORT instruction
ssa_gpr->REG(ip) += 3; // Skip ENCLU, which is always a 3-byte instruction
ssa_gpr->REG(flags) |= 1; // Set CF to indicate error condition, see implementat
is_exception_handled = true; // The exception has been handled in the 1st phase
goto handler_end;
}
if (size_t(&Leverifyreport2_inst) == ssa_gpr->REG(ip) && SE_EVERIFYREPORT2 == ss
{
// Handle the exception raised by everifyreport2 instruction
ssa_gpr->REG(ip) += 3; // Skip ENCLU, which is always a 3-byte instruction
ssa_gpr->REG(flags) |= 64; // Set ZF to indicate error condition, see implementa
ssa_gpr->REG(ax) = EVERIFYREPORT2_INVALID_LEAF;
is_exception_handled = true; // The exception has been handled in the 1st phase
goto handler_end;
}
```

핵심 분석 포인트:

- **명령어 포인터 조정**: ENCLU는 항상 3바이트 명령어이므로 IP += 3

- **플래그 설정**: CF(Carry Flag) 또는 ZF(Zero Flag)로 에러 상태 표시
- **에러 코드 설정**: RAX 레지스터에 구체적인 에러 코드 저장
- **예외 처리 완료**: 1차 예외 핸들러에서 처리 완료 표시

3.2 SGX 에러 코드 체계

파일: `common/inc/sgx_error.h`

```
typedef enum _status_t {
SGX_SUCCESS = 0x00000000,
/* General errors */
SGX_ERROR_UNEXPECTED = 0x00000001,
SGX_ERROR_INVALID_PARAMETER = 0x00000002,
SGX_ERROR_OUT_OF_MEMORY = 0x00000003,
/* ENCLU instruction errors */
SGX_ERROR_INVALID_FUNCTION = 0x00001001,
SGX_ERROR_OUT_OF_TCS = 0x00001003,
SGX_ERROR_ENCLAVE_CRASHED = 0x00001006,
/* ENCLS instruction errors */
SGX_ERROR_INVALID_ENCLAVE = 0x00002001,
SGX_ERROR_INVALID_SIGNATURE = 0x00002003,
SGX_ERROR_OUT_OF_EPC = 0x00002005,
} sgx_status_t;
```

3.3 시뮬레이션 모드 에러 처리

파일: sdk/simulation/tinst/t_instructions.cpp

```
uintptr_t  _SE3(uintptr_t  xax, uintptr_t  xbx, uintptr_t  xcx,
uintptr_t  xdx, uintptr_t  xsi, uintptr_t  xdi)
{
switch (xax)
{
case SE_EEXIT:
_EEXIT(xbx, xcx, xdx, xsi, xdi);
// never reach here
return 0;

case SE_EGETKEY:
return _EGETKEY(reinterpret_cast<sgx_key_request_t *>(xbx),
reinterpret_cast<uint8_t *>(xcx));

case SE_EREPORT:
_EREPORT(reinterpret_cast<sgx_target_info_t*>(xbx),
reinterpret_cast<sgx_report_data_t*>(xcx),
reinterpret_cast<sgx_report_t*>(xdx));
return 0;
}
```

```
GP(); // General Protection Fault

return (uintptr_t)-1;

}
```

핵심 분석 포인트:

- **함수 디스패치**: leaf 번호(RAX)에 따른 적절한 시뮬레이션 함수 호출
- **매개변수 변환**: 레지스터 값을 적절한 포인터 타입으로 캐스팅
- **에러 시뮬레이션**: 잘못된 leaf 번호 시 GP(General Protection) 예외 발생

4. 플랫폼별 구현 차이 분석

4.1 Linux vs Windows 레지스터 규약

Linux (System V ABI):

```
#ifdef LINUX64

# define naked_arg0 %rdi

# define naked_arg1 %rsi

# define naked_arg2 %rdx

# define naked_arg3 %rcx

#endif
```

Windows (Microsoft x64 ABI):

```
#ifdef WIN64

# define naked_arg0 %rcx
```

```
# define naked_arg1 %rdx

# define naked_arg2 %r8

# define naked_arg3 %r9

#endif
```

4.2 스택 정렬 차이

Linux:

```
#ifdef LINUX64

sub $(10*SE_WORDSIZE), %xsp /* 16-byte alignment */

#endif
```

Windows:

```
#ifdef WIN64

sub $(8*SE_WORDSIZE), %xsp /* 16-byte alignment + shadow space */

#endif
```

4.3 예외 처리 메커니즘 차이

Linux - Signal Handler:

```
static void sig_handler(int sig, siginfo_t* info, void* context)

{

ucontext_t* uc = (ucontext_t*)context;
```

```
if (sig == SIGSEGV) {

// Handle segmentation fault

uc->uc_mcontext.gregs[REG_RAX] = SE_EENTER;

uc->uc_mcontext.gregs[REG_RDI] = ECMD_EXCEPT;

}

}
```

Windows - Structured Exception Handling:

```
LONG WINAPI sgx_exception_handler(EXCEPTION_POINTERS* exception_pointers)

{

CONTEXT* context = exception_pointers->ContextRecord;

if (exception_pointers->ExceptionRecord->ExceptionCode == EXCEPTION_ACCESS_VIOLA

// Handle access violation

context->Rax = SE_EENTER;

context->Rdi = ECMD_EXCEPT;

return EXCEPTION_CONTINUE_EXECUTION;

}

return EXCEPTION_CONTINUE_SEARCH;

}
```

4.4 시스템 호출 인터페이스 차이

Linux - IOCTL:

```
int sgx_create_enclave(int fd, struct sgx_enclave_create* create_param)
{
return ioctl(fd, SGX_IOC_ENCLAVE_CREATE, create_param);
}
```

Windows - DeviceIoControl:

```
BOOL sgx_create_enclave(HANDLE device, PSGX_ENCLAVE_CREATE create_param)
{
DWORD bytes_returned;
return DeviceIoControl(device, IOCTL_SGX_ENCLAVE_CREATE,
create_param, sizeof(*create_param),
NULL, 0, &bytes_returned, NULL);
}
```

5. 성능 최적화 기법

5.1 레지스터 사용 최적화

```
/* 레지스터 보존 최소화 */
DECLARE_LOCAL_FUNC do_egetkey
SE_PROLOG /* 필요한 레지스터만 보존 */
mov $SE_EGETKEY, %xax /* leaf 번호 설정 */
```

```
ENCLU /* 명령어 실행 */

SE_EPILOG /* 레지스터 복원 */

END_FUNC
```

5.2 분기 예측 최적화

```
#ifdef SE_SIM

cmp $SGX_SUCCESS, %xax /* 시뮬레이션 모드 */

jnz .Legetkey_done /* 실패 시 분기 */

#else

jz .Legetkey_done /* 하드웨어 모드: 성공 시 ZF=1 */

#endif
```

5.3 캐시 최적화

```
/* YMM 레지스터 상위 비트 클리어로 AVX-SSE 전환 페널티 방지 */

lea_symbol g_clean_ymm, %xbx

movl (%xbx), %ecx

cmpl $0, %ecx

je 1f

vzeroupper /* YMM 상위 128비트 클리어 */

1:
```

6. 보안 고려사항

6.1 정보 누출 방지

```
/* EEXIT 시 모든 레지스터 클리어 */
xor %xcx, %xcx
xor %xdx, %xdx
#ifdef LINUX64
xor %r8, %r8
xor %r9, %r9
/* ... 모든 범용 레지스터 클리어 ... */
#endif
```

6.2 스택 보호

```
/* 스택 카나리 검사 */
int check_static_stack_canary(void *tcs) {
size_t *canary = TCS2CANARY(tcs);
if (*canary != (size_t)__stack_chk_guard) {
return -1; /* 스택 오버플로우 감지 */
}
return 0;
}
```

6.3 메모리 경계 검사

```
/* Enclave 메모리 경계 검사 */
int sgx_is_within_enclave(const void *addr, size_t size) {
size_t start = (size_t)addr;
size_t end = start + size;
if (start >= end) return 0; /* 오버플로우 검사 */
return (start >= g_enclave_base &&
end <= g_enclave_base + g_enclave_size);
}
```

결론

SGX 하드웨어 명령어의 소프트웨어 래퍼 구현은 다음과 같은 핵심 특징을 가집니다:

1. **이중 구현**: 하드웨어 모드와 시뮬레이션 모드를 모두 지원
2. **플랫폼 추상화**: Linux/Windows 간의 차이를 매크로와 조건부 컴파일로 처리
3. **성능 최적화**: 레지스터 사용 최소화, 분기 예측 최적화, 캐시 친화적 구현
4. **보안 강화**: 정보 누출 방지, 스택 보호, 메모리 경계 검사
5. **에러 처리**: 하드웨어 예외를 소프트웨어 에러 코드로 체계적 변환

이러한 구현을 통해 SGX SDK는 복잡한 하드웨어 기능을 개발자가 쉽게 사용할 수 있는 고수준 API로 제공하면서도, 보안성과 성능을 모두 보장합니다.